

Mining Attention Dynamics: Auditing KV-Saliency Prediction in LLMs

Ziqing Li

Huazhong University of Science and Technology
Wuhan, China
d202381502@hust.edu.cn

Kaicheng Tang

The Chinese University of Hong Kong
Hong Kong, China
kctang0410@link.cuhk.edu.hk

Jiawei Guo

Beihang University
Beijing, China
jwguo@buaa.edu.cn

Jianxi Chen*

Huazhong University of Science and Technology
Wuhan, China
chenjx@hust.edu.cn

Abstract—A scorer that predicts which cached attention keys and values (KV) to retain can be ranked by a metric pooled over blocks from many retention decisions, whereas a selector acts on one decision’s candidate set and budget. KVSaliencyBench records candidate sets, future-attention labels, source-prompt identities and versioned feature realizations, and scores pooled ranking together with selection under per-decision budgets. On traces of two instruction-tuned models collected with a 4K-token input cap, a three-parameter logistic scorer and LightGBM reach pooled area under the receiver operating characteristic curve (AUC) 0.890 and 0.896. Yet a within-layer attention signal recovers 0.659 of each decision’s future top-decile blocks, against 0.613 and 0.615 for those models. An exact pair decomposition and a scaling of the scorer’s cross-layer coefficient show that better cross-decision ordering coexists with worse within-decision ranking and top-decile recall; the reversal holds on a source-disjoint split and narrows at 20% retention. A separate pinned masked-loop evaluation finds two complete runtime policies equivalent in mean LongBench F1 within a practical margin of two absolute points over seven datasets, both below full cache. A physical reference backend stores only the retained KV but does not establish numerical equivalence with masking. These results support evaluating a retention scorer at its own candidate set and budget, then checking task quality on the policy’s own trajectory; they make no serving or per-task equivalence claim.

Index Terms—KV cache, attention prediction, empirical audit, calibration, benchmarking, closed-loop evaluation.

I. INTRODUCTION

Cached attention keys and values (KV state) grow with context while attention concentrates on few positions. KV-management methods exploit this through accumulated attention [1], attention sinks with recent windows [2], recent-query observations [3], query-key page bounds [4], cross-layer information [5] and learned retention [6]. Most rest on a prediction—that the blocks kept are the ones future queries will attend to—and learned predictors now target attention directly [7]. What is rarely separated is pooled ranking, the per-decision budget a selector imposes, the features a runtime can reconstruct, the candidate set and task quality. Methods are

mostly compared at the end of the pipeline, on answer scores, where the scorer, the budget rule, the features and the feedback trajectory all move together, so a gain cannot be attributed to any one. Better prediction need not identify a better retention policy: selection changes later model states and their outputs.

Measuring the prediction itself needs objects a task benchmark does not record: the candidate set and budget of each retention decision, the features available at runtime, and the future attention the decision is betting on. KVSaliencyBench records them, so that a gain can be attributed to prediction rather than to a budget rule and a proxy ranking checked against task quality.

Contributions. (1) *A protocol and benchmark.* KVSaliencyBench fixes future-attention labels, selector semantics (Table I), request- and source-prompt splits, tie-aware per-decision metrics and versioned collection; its default entry point (version 2.0) scores pooled and per-decision metrics together from a prediction table alone and verifies candidate sets against a manifest when one is supplied. (2) *Controlled findings.* A three-parameter logistic scorer on two attention-magnitude views is within 0.006 AUC of LightGBM [8] on v2 traces (0.008 at most across tree capacities) without a calibration step, yet at the per-decision top-decile budget the within-layer signal alone recovers more future top-decile blocks than either, on request- and source-disjoint splits, a reversal that shrinks at 20% retention; an exact split shows pooled AUC is a cross-decision statistic, which the scorer’s binary cross-layer term raises while lowering within-decision AUC and recall. A pinned bfloat16 masked-loop rerun finds the reconstructed scorer and the H2O-style accumulator mean-F1 equivalent within a practical ± 0.02 margin over seven datasets (two one-sided tests, TOST, $p = 0.0023$), and a bridge measures reference-backend feature reconstruction (§IV-B). (3) *An audit and its repair.* We diagnose corrupted float16 Qwen2.5 generation, rerun with a pinned simulator, audit capacity effects from logged counts, tie rules and scorer provenance, and report physical-KV parity levels.

*Corresponding author: Jianxi Chen (chenjx@hust.edu.cn).

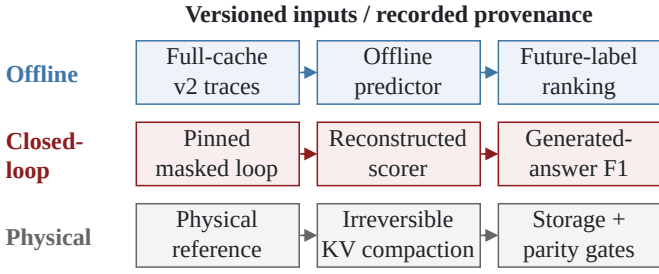


Fig. 1. The three evaluation paths, one per row: offline prediction (blue), closed-loop task quality (red) and the physical reference (grey); their selector semantics differ (Table I).

II. PROBLEM AND EVIDENCE LEVELS

A. Attention-defined saliency

At layer ℓ and decode step t , let $\mathcal{B}_{\ell,t}$ contain the logical KV blocks currently under consideration. A block groups contiguous original token positions. We aggregate attention by averaging over heads and over a block’s token positions; denote the resulting statistic by $a_{\ell,b,t}$. It is a block-mean attention statistic, not necessarily a mass normalized to sum to one over blocks. For a horizon h and a fraction r , the target is $z_{\ell,b,t}^{(h)} = 1[b \in \text{Top}_{\lceil r|\mathcal{B}_{\ell,t}| \rceil} \{a_{\ell,b',t+h} : b' \in \mathcal{B}_{\ell,t}\}]$; ranking is restricted to blocks present at t . We use $r = 0.1$ and report $h = 4$; horizons 1, 16 and 64 are recorded only.

A scorer $f(\mathbf{x}_{\ell,b,t})$ estimates the label probability from recorded features; positives are a small fraction of blocks, and serial dependence means millions of block rows are not millions of independent observations.

B. Three different evaluation targets

Fig. 1 locates the evaluation paths. Offline prediction scores future-attention labels on fixed full-cache traces; served-oracle diagnostics score a current-attention target along the evaluated trajectory, and attention importance is not answer correctness; closed-loop task quality scores answers the selector’s own decisions helped produce, here through masking rather than a cache manager. A fraction of blocks allowed to attend is a *logical retention budget*, not of device memory.

C. Statistical protocol

Offline ranking uses AUC, threshold-based AUPRC, precision/recall at $k = r$, Brier score and, for scorers that emit probabilities only, expected calibration error (ECE). Pooled metrics rank rows *across* decisions; top- k metrics are reported pooled over test rows *and* per cache decision (request, layer, step) at the label’s own $\lceil rn \rceil$, averaged over decisions (decision-macro). With n_d^+ positives and n_d^- negatives in decision d , pooled AUC splits exactly by pair origin:

$$\text{AUC}_{\text{pool}} = \omega \text{AUC}_{\text{in}} + (1 - \omega) \text{AUC}_{\text{x}}, \quad (1)$$

where $\omega = \sum_d n_d^+ n_d^- / (\sum_d n_d^+ \sum_d n_d^-)$ is the share of same-decision (positive, negative) pairs, AUC_{in} the AUC over them (pair-weighted, not a mean of per-decision AUCs) and AUC_{x} that over cross-decision pairs. Training and testing use disjoint

requests and paired bootstrap intervals resample requests, not block rows. Unless marked otherwise, intervals are 95%; equivalence intervals are 90%. A 10,000-draw sensitivity analysis jointly resamples held-out model-requests that share a source prompt, on both splits. Top- k score ties use a stable descending sort (row order: the block index inside a decision), and v2 label boundaries keep the collector’s `argpartition` rule with exactly $\lceil rn \rceil$ positives; the artifact states both orders exactly. Taking instead the exact expectation under uniform random tie-breaking changes the decision-macro recall of Table II’s three main scorers by less than 2×10^{-5} at 10% and 20% retention, and alters no other printed cell but the age proxy’s: 99.9% of its rows tie, so row order picks its blocks (0.208), and the table prints the expectation, 0.109. Label ties are not randomized. The main downstream comparison aggregates paired request scores into architecture–dataset cells and averages the two architectures within each dataset; a TOST analysis [9], [10] asks whether the mean difference over the seven datasets lies inside a practical margin. Failing to detect a difference is not equivalence, and two comparisons equivalent to one baseline need not be equivalent to each other.

III. SIGNALS AND EMPIRICAL FEATURE SELECTION

The feature family contains a normalized within-layer exponential moving average (EMA) of attention $x_{w1} = \bar{a}_{\ell,b,t} / \max_{b'} \bar{a}_{\ell,b',t}$, a previous-layer hot-block indicator $x_{cl} = 1[\bar{a}_{\ell-1,b,t} \text{ is in the top-}r]$, a query-derived proxy $x_q = (1 + \cos(\tilde{q}_t, K_{\ell,b})) / 2$ and a temporal-age feature $x_{age} = \exp(-(t - p_b)/w)$, $w = 64$ steps. Here $\bar{a}$ is an EMA and p_b the block-creation step, so x_{age} is an age proxy, not an LRU access-recency measurement; the tildes mark the archived query representation, whose coordinate-space audit in §IX limits conclusions about that view. The cross-layer signal is an attention-magnitude carry-over, not InfiniGen’s query-based mechanism; at layer 0 the offline extractor takes the same top- r indicator from the layer’s own EMA, where runtime realizations need explicit fallback and alignment rules (Table I).

We estimate relevance $I(X_i; Z)$ with quantile-binned plug-in estimators, in nats; the version-2 estimates are in §V-A and raw-probe AUCs in Table II. Archived estimates of the conditional information $I(X_i; Z | X_{-i})$, grid-sensitive for the cross-layer view, agree that the magnitude views carry what the proxies do not. They motivated the two-view subset; they are not a sufficiency certificate, and the evidence for the compact model is its measured predictive performance.

IV. PREDICTOR FAMILY AND FEATURE REALIZATIONS

A. A compact offline scorer

We fit a ridge-regularized logistic model by Newton–IRLS, $f(\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$; on the within-layer and cross-layer features it has two coefficients and one intercept (ridge 10^{-3} on the coefficients only, unstandardized features, at most 40 iterations; `docs/FROZEN_CONFIG.md` lists every frozen constant of the fits, features and splits). A log-loss fit without

class reweighting has low measured ECE here and the class-balanced LightGBM does not, but an unweighted or recalibrated tree removes that difference (§V): calibration, which only a thresholded selector needs, is separate from ranking here.

B. Offline predictor versus runtime reconstruction

The masking-loop scorer reconstructs the cross-layer view from the previous layer’s EMA, neutralizes the query view, and feeds selection into later states, so its task F1 cannot inherit the offline predictor’s AUC. Table I distinguishes fixed offline features, policy-dependent runtime features and the irreversible physical reference. **Bridge experiment.** With the frozen archived two-view checkpoint, one label rule (top- $[0.1n]$ of block attention four steps ahead among the blocks present now and still present then) and the 128 frozen prompts of §VI, we score (A) offline-style features re-derived from the full-cache attention stream under version-2 conventions, (B) the reference backend’s runtime realization on the same rows, and (C) that realization along the on-policy trajectory at 20% budget. In A and B the candidates are every block present; in C they are the blocks retained now *and* at $t + h$, a survivor-conditioned arm whose n and label denominator are taken among those survivors. Mean per-request AUC is 0.902/0.899/0.989 on Llama-3.1-8B (bfloat16) and 0.894/0.892/0.989 on Qwen2.5-7B (float32): the reference backend’s feature realization costs -0.003 $[-0.004, -0.001]$ (84/128 requests negative) and -0.002 $[-0.003, -0.0004]$ (75/128), whereas the on-policy arm is higher by $+0.090$ and $+0.098$ (128/128 both). That is not evidence that retention makes prediction easier: C ranks only blocks the policy already kept, so an evicted block can never be missed, and the value neither isolates a feedback effect nor measures the coverage lost to eviction. Neither contrast reproduces the archived 0.948-to-0.643 drop of §VIII, which also differs in corpus, aggregation and numerics; attribution would need a label-only arm on identical trajectories, candidates, features and scorer, so that gap stays unexplained.

C. Selector semantics across the three evidence objects

Table I lists the rules that decide what each object measures. Two differ on purpose: the masked loop (pinned-rerun rules) recomputes its kept set every eight steps, so an excluded block can return, and budgets $\text{round}(0.2n)$ blocks with n counting the blocks of generated tokens too, whereas the reference deletes irreversibly and fixes its budget from the prompt.

V. PREDICTION MEASUREMENTS

Data and comparators. Corrected version-2 traces carry the offline evidence; version-1 diagnostics are in §VIII. Single-signal rows score one recorded feature and do not reimplement the papers that motivate them; the learned comparators are LightGBM on all four views (150 trees of depth 3) and the two-view logistic fit, with larger logistic, MLP and online fits left to the artifact. Every fit, including the isotonic control’s calibration split, uses held-in requests only, and no held-out

TABLE I
SELECTOR AND LABEL SEMANTICS. BLOCKS ARE 32 TOKENS; BUDGETS COUNT BLOCKS, MANDATORY ONES INCLUDED; n IS THE CANDIDATE COUNT, L THE PROMPT LENGTH, K THE SERVED-ORACLE SIZE OF EXP#7. A LOOP BUDGET BELOW THE MANDATORY SET (7 BLOCKS ON ONE PROMPT PER MODEL) KEEPS RECENT BLOCKS, THEN SINKS.

rule	v2 collector	masked loop	physical reference
attention	head/token mean; ragged block over members	head-group/query mean; ragged block padded	head/query/token mean
EMA	decay 0.9 from first decode value	H2O-style: last-32 sum; reconstructed: last-8 EMA, 0.9	decay 0.9; last-64 prefill queries
within view cross view (0/1)	$\bar{a}/(\max \bar{a} + 10^{-9})$ in top- $[0.1n]$ of previous-layer EMA; layer 0: of own EMA	$\bar{a}/(\max \bar{a} + 10^{-9})$ same rule; layer 0: own EMA	$\bar{a}/\max \bar{a}$ same rule; layer 0: zero
query / age	post-RoPE q ; age from block creation	query 0.5; age since last top- k	unused
label	future top- $[0.1n]$, real lookahead	current unmasked top- K (Exp#7)	unused
budget	—	$\text{round}(0.2n)$, every 8 steps	$[0.2\lceil L/32 \rceil]$, prompt-fixed
mandatory	—	reconstructed: 4 sink $[2] + 4$ recent; H2O-style: half heavy, half recent	1 sink + 1 recent
re-admission, new tokens	—	allowed at each decision; new tokens unmasked, budget grows	irreversible; new tokens eligible, fixed budget

request enters a fit or a threshold; the two-view subset was fixed from version-1 diagnostics, so the version-2 relevance estimates below are descriptive.

A. Exp#1: Ranking and calibration

Version-2 corpus. Table II uses checksum-verified traces of the same 128 LongBench QA prompts (64 Qasper, 64 NarrativeQA) for Llama-3.1-8B and Qwen2.5-7B: 30.4M rows and 256 model-requests. Pooled metrics use 120K training and 150K held-out rows; per-decision metrics use every held-out row. Refitting both models on the same split, seed and test rows at up to all 22.8M held-in rows (190-fold) lifts LightGBM by 0.0005 AUC (0.8961 to 0.8966), widens the paired deficit only from -0.0057 to -0.0062 $[-0.0073, -0.0051]$, and its per-decision recall holds at 0.615, so the gap is stable to training size; six LightGBM capacities (depth 3–8, 150–600 trees, one leaf-limited, balanced or not) at 120K and 1.92M rows keep its AUC lead between 0.0005 and 0.0076 and its per-decision recall between 0.615 and 0.622, never reaching the raw EMA’s 0.659. The split holds out 64 model-requests, so Table II measures transfer to new model-requests, not to unseen text: 50 of their 57 source prompts also occur in training through the other model. A source-disjoint split (the same 32 prompts held out for both models; the benchmark’s default) agrees within 0.006 AUC for every method and reproduces every entry of the lower block within 0.004 (top-decile recall 0.656, 0.610 and 0.612). The two-view fit reaches AUC 0.890 (source-prompt bootstrap

[0.882, 0.898]) against LightGBM’s 0.896, a paired deficit of -0.0057 $[-0.0068, -0.0046]$, larger than the version-1 -0.0010 but still small. At the per-decision operating point the ordering changes: the raw within-layer EMA recovers 0.659 of each decision’s future top-decile blocks against 0.613 for the two-view fit and 0.615 for LightGBM, though its pooled top-decile precision (0.542) and AUC (0.873) are lower. The decision-macro gaps are $+0.046$ $[0.042, 0.049]$ and $+0.044$ $[0.041, 0.047]$ (source-cluster bootstrap of the decision-macro statistic, 10,000 joint draws of 57 source prompts), and $+0.046$ $[0.043, 0.049]$ and $+0.045$ $[0.042, 0.048]$ on the source-disjoint split; the gap is positive within each of the 64 held-out requests on both, and the ordering holds in each model (recall 0.652, 0.616 and 0.616 on Llama; 0.665, 0.610 and 0.614 on Qwen). The reversal’s size depends on the operating point: at the closed loop’s 20% budget, per-decision recall is 0.840, 0.838 and 0.830 and the gaps fall to $+0.002$ $[0.002, 0.003]$ (positive in 52/64 requests) and $+0.010$ $[0.009, 0.011]$, consistent with, but not explaining, Exp#8’s mean-F1 equivalence.

Where the reversal comes from. Within one decision the within-layer view is a monotone rescaling of the raw EMA (its denominator $\max_{b'} \bar{a}_{\ell, b', t}$ is shared by the candidates), so $\sigma(w_1 x_{w1} + w_2 x_{c1} + b)$ reorders the raw view through exactly one term, the *binary* cross-layer indicator: a fixed offset $w_2/w_1 = 0.025$ ($w_1 = 97.7$, $w_2 = 2.46$) against a corpus mean x_{w1} of 0.015. The lower block of Table II evaluates Eq. (1) on every held-out row. Its weight ω depends on the decision sizes, not on the scorer: 0.0016% here, about one over the 61,184 decisions. That pooled AUC is a cross-decision statistic is therefore arithmetic; the finding is what one fitted term does to the two parts. The indicator raises pooled AUC by $+0.017$ $[0.015, 0.019]$ but lowers pair-weighted within-decision AUC by 0.0025 $[0.0023, 0.0028]$ and top-decile recall by 0.046; scaling the offset by λ (Fig. 2(a); $\lambda = 0$ is the raw EMA order, $\lambda = 1$ the fit) shows one eighth of the offset already gives 83% of the first and 65% of the second. At the top-decile boundary the term swaps in 22% of the selected blocks, all previous-layer-hot: 20% are positive, against 41% of the blocks they displace. The indicator is informative overall (61% of previous-layer-hot blocks are future top-decile blocks, 4% of the rest) but not at the margin; every decision holds exactly $\lceil 0.1n \rceil$ positives, so it cannot be flagging salient-rich decisions, and why it makes scores comparable across decisions is untested. LightGBM moves the same way ($+0.022$ pooled AUC, -0.005 within-decision AUC, -0.044 recall).

Relevance and reliability: the magnitude views carry $I(X_t; Z)$ of 0.104 and 0.099 nats, the query proxies 0.007 (cosine) and 0.025 (dot-max), the age proxy under 0.001, on a 400K-row sample of all pooled rows (from held-in rows only they move by at most 0.0011). The class-balanced LightGBM has ECE 0.208 (Fig. 2(b)) while the unweighted (0.005) and isotonic (0.003) variants match the two-view fit (0.006). The isotonic control reserves 48 of the 192 training model-requests, its trees use the other 144, and the 64 test requests are excluded from both fitting stages: request-level calibration separation, not source-level independence.

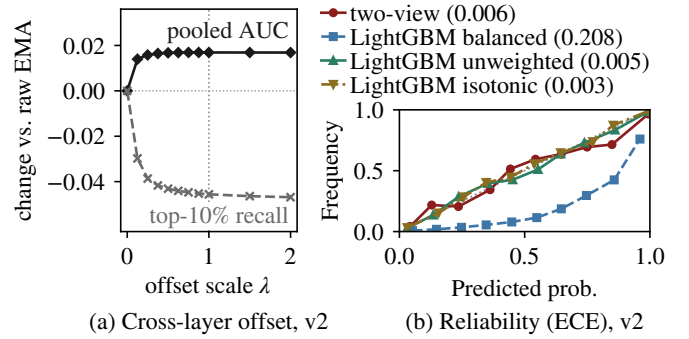


Fig. 2. Exp#1, version-2 corpus. (a) The fitted cross-layer offset scaled by λ from 0 (raw within-layer EMA order) through 1 (the fit) to 2, on every held-out row: pooled AUC and per-decision top-decile recall as changes from the raw EMA. (b) Reliability of the two-view fit and of class-balanced, unweighted and isotonic LightGBM in 10 equal-width bins; ECE in parentheses.

TABLE II
EXP#1: VERSION-2 RANKING AND CALIBRATION, HORIZON 4, REQUEST SPLIT.

method	AUC	AP	P@k pool	P@k dec.	ECE
age proxy	.502	.102	.102	.109 [†]	—
query proxy (cosine)	.599	.135	.147	.415	—
prev-layer indicator	.783	.412	.609	.610	—
within-layer EMA	.873	.609	.542	.659	—
LightGBM, balanced	.896	.674	.626	.615	.208
two-view logistic, refit	.890	.667	.624	.613	.006
two-view, archived weights	.890	.667	.622	.611	.019
method	pooled	within	cross	rec. 10%	rec. 20%
within-layer EMA	.873	.930	.873	.659	.840
two-view logistic, refit	.890	.928	.890	.613	.838
LightGBM, balanced	.895	.925	.895	.615	.830

Upper block, 150K sampled held-out rows: AP is tie-aware AUPRC; ECE only for probability outputs; P@k dec. is top-decile precision (equal to recall there), decision-macro over the 61,184 decisions of all 7.6M held-out rows. [†]Expectation under random tie-breaking (.208 by row order, §II). Lower block, every held-out row (LightGBM .895 pooled against .896 on the sample): pair-weighted AUC over within- and cross-decision pairs (Eq. (1)); decision-macro recall. Further rows are in the artifact.

VI. PHYSICAL-KV REFERENCE AND RUNTIME SCOPE

Scope. Two boundaries hold for every runtime number in this paper. (i) Masking is not physical eviction: equal outputs under physical eviction would need matched token histories, kept sets, position semantics, feature availability and re-admission rules, none of which the simulator establishes, so every closed-loop task-quality result is scoped to the masked loop (and a logical budget, §II, releases no device memory). (ii) Nothing here is a serving result: kept-set sizes and allocator peaks measure none of production per-token latency, GPU-memory release, contention or throughput, and the reference runner implements no paging, offload or batching.

Exp#6: Physical-KV implementation check. An eager, single-request reference backend with original token positions, irreversible block eligibility and independent compact K/V storage is compared with a dense masked reference under the same eligibility rule (Table I); gates compare native full-cache logits with the no-eviction path and replay a frozen teacher

TABLE III

Exp#6 (PHYSICAL-KV CHECK) AT 20% BLOCK BUDGET. F1: OFFICIAL LONGBENCH SCORER, ALL REFERENCES. STORED: MEAN UNIQUE K/V TENSOR STORAGE (MiB) AFTER THE LAST STEP; LIVE: MEAN RETAINED PAYLOAD (MiB).

realization	Llama, bfloat16			Qwen, float32		
	F1	stored	live	F1	stored	live
full physical	.223	496	496	.246	434	434
H2O-style mask	.183	496	94	.233	434	82
H2O-style physical	.181	94	94	.233	82	82
reconstructed mask	.177	496	94	.232	434	82
reconstructed physical	.178	94	94	.232	82	82

schedule between mask and compact storage. H2O-style here means a shared-head block accumulator, not H2O.

Table III reports both models on one A100-80GB, Transformers 4.51.3, greedy decoding, the same 128 English QA prompts (64 Qasper, 64 NarrativeQA), input cap 4096, at most 128 new tokens and middle truncation on 104/128 prompts. Llama-3.1-8B-Instruct runs in bfloat16; Qwen2.5-7B bfloat16 parity exceeded the preset tolerance (maximum absolute error 0.25 versus 0.15 plus 2% of the logit) while argmax agreed, so Qwen is reported in float32. At 20% budget compact storage equals the live payload whereas the mask keeps full-size tensors, a $5.3\times$ ratio for both. Greedy sequences match between the two paths on 98/128 and 100/128 Llama requests (mean absolute F1 differences 0.014 and 0.007) and 128/128 Qwen requests. Allocator decode peaks fall only from 15.51 to 15.10 GiB (Llama) and 29.05 to 28.63 GiB (Qwen) because weights dominate, with prefill-plus-first-compaction peaks at 15.77 and 29.23 GiB. These free-running agreements are the weakest of three parity levels: the teacher-forced replay of §IX passes at 1024 tokens and four steps but exceeds its tolerance over 40 evictions, so the table supports the storage observation, not serving equivalence.

Different cohorts, same scorer. Table III uses Table IV’s scorer, the official LongBench F1 with the maximum over references (rescoring its stored generations with Exp#8’s code changes no request), but another cohort: two of Exp#8’s seven datasets (Qasper and NarrativeQA, full-cache F1 0.41 and 0.19 there against 0.419 over all seven), a different row sample, another prompt template and a 128- instead of 48-token generation cap. Compare within a table only.

VII. CLOSED-LOOP TASK QUALITY

Exp#8: Matched-retention answer quality. The policy evaluated here is the runtime reconstruction of §IV-B, not the offline predictor of Table II. The main comparison uses chat-formatted LongBench QA [11]: its seven English question-answering datasets (NarrativeQA, Qasper, MultiFieldQA-en, HotpotQA, 2WikiMultihopQA, MuSiQue, TriviaQA; abbreviated in Fig. 3), two architectures (Llama-3.1-8B and Qwen2.5-7B) and 448 source prompts, 64 per dataset, each evaluated on both architectures (896 model-prompt evaluations, 64 per architecture-dataset cell), at 20% logical retention, each policy driving its own masked decoding trajectory.

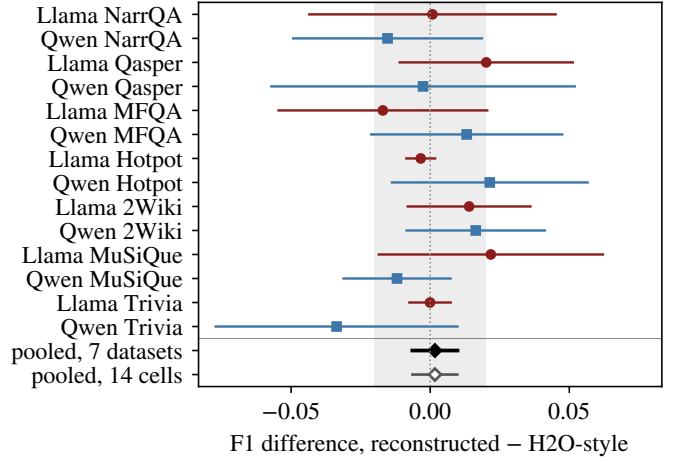


Fig. 3. Exp#8, pinned bfloat16 rerun. Reconstructed-minus-H2O-style F1 per architecture $\times$ dataset cell (64 paired requests, 95% t intervals; the two models of a dataset adjacent) and pooled estimates (90% t intervals): seven datasets (primary, filled) and 14 cells (sensitivity, hollow); the band is the ± 0.02 margin.

TABLE IV

Exp#8: MASKED-LOOP LONGBENCH F1 AT 20% RETENTION, 896 MODEL-PROMPT EVALUATIONS. Δ IS THE DIFFERENCE FROM THE H2O-STYLE ACCUMULATOR WITH A 90% t INTERVAL OVER THE SEVEN DATASETS (DF 6); p_{TOST} IS THE TOST p AT ± 0.02 OVER THE SAME DATASETS (NONE FOR FULL CACHE: THE QUESTION THERE IS THE LOSS). PINNED BFLOAT16 RERUN, OFFICIAL SCORER, ALL REFERENCES.

policy realization	F1	Δ [90% CI]	p_{TOST}
full cache	.419	+0.047 [+0.015, +0.078]	—
H2O-style	.372	—	—
reconstructed	.374	+0.002 [-0.006, +0.010]	.0023
Ada-KV-style	.363	-.008 [-0.016, -.0004]	.015
PyramidKV-style	.349	-.023 [-0.046, +0.001]	.582

Inferential unit and margin. Because the two architectures answer the same questions, the 14 cells are not independent. The primary analysis averages the two models within each dataset and runs paired TOST over the seven datasets (df 6); the 14 cells treated as independent are a sensitivity analysis. The margin is two absolute F1 points, 5.4% of the rerun baseline 0.372: a practical tolerance carried over from the archived analysis, not pre-registered and not a deployment threshold. Fig. 3 exposes task heterogeneity: mean equivalence is not per-task.

Pinned rerun. With the official LongBench scorer and all references, the reconstructed scorer’s mean F1 difference from the accumulator over 896 evaluations is +0.0017: the seven-dataset TOST gives $p = 0.0023$ at ± 0.02 and the 14-cell sensitivity analysis $p = 0.0007$. The seven-dataset t interval is $[-0.0064, 0.0098]$; the 14-cell t -based and cell-bootstrap 90% intervals are $[-0.0063, 0.0097]$ and $[-0.0056, 0.0087]$. Per architecture the difference is +0.0052 (Llama) and -0.0018 (Qwen), each inside the same ± 0.02 margin over its seven datasets ($p_{\text{TOST}} = 0.016$ and 0.026 : equivalence tests, not tests of a gain or a loss). The Ada-KV-style comparator is also

equivalent (-0.0084 , $p_{\text{TOST}} = 0.015$); for the PyramidKV-style one equivalence is not established (-0.023 , $p_{\text{TOST}} = 0.58$), which does not show it is worse. Both allocation comparators only move a fixed total budget across layers (Ada-KV-style: by attention concentration; PyramidKV-style: a linear pyramid) and select within a layer as H2O-style does, so no finding covers original H2O, Ada-KV or PyramidKV, or equivalence between the reconstructed and Ada-KV-style policies; and this is equivalence between compressed policies, not absence of compression loss: both stay 0.045–0.047 below full cache. The policies also differ in EMA window and mandatory blocks (Table I): this compares two complete runtime realizations, not a scorer-only intervention, which is untested. Logged selected-set sizes (the blocks chosen at the last decision, layer mean, per decode step) average 25.02 for the H2O-style, reconstructed and Ada-KV-style policies and 25.01 for the PyramidKV-style one (median 26, range 7–26): the policies are matched in selected blocks, not only in the nominal 0.2. Tokens decoded since the last decision stay unmasked and tail blocks are ragged, so equal per-step active-token counts are not claimed.

What the rerun repaired. Archived Qwen float16 decoding emitted all-NaN logit rows, producing token 0 (“!”): a 56-prompt replay found this in 54 requests, against none in bfloat16/float32. In the rerun no Qwen full-cache output contains a run of “!” (0/448 versus 181/448) and 34/448 reach the 48-token cap (versus 210/448). The archived F1 difference was $+0.00511$ under a single-reference scorer ($+0.0030$ after all-reference rescaling).

Exp#9: Archived budget and context checks. An archived retention sweep (0.10 to 0.50) resolves no per-budget quality gain, and at 16K a 448-prompt Llama sample has mean F1 difference -0.0006 (request-level 95% interval $[-0.012, 0.012]$) where a 160-prompt pilot had suggested $+0.010$: we claim no long-context gain, and neither check proves curve equivalence.

VIII. HISTORICAL DIAGNOSTICS

Every measurement here comes from the unpinned version-1 pipeline, whose defects—float16 Qwen2.5 NaN logits, a query extractor that never formed the attention-space product, an AUPRC that credited tied scores for row order—are in §IX. Its raw rows are gone, so none of these numbers is version-2 evidence or can be recomputed. They motivated the audit, and docs/HISTORICAL_DIAGNOSTICS.md lists each with its source. On 43,931,784 rows from four instruction-tuned families the two-view fit trailed LightGBM by 0.0010 AUC (0.948 against 0.949), the ordering the corrected corpus shows at a wider margin; the workload-extension, transfer, online-update, risk-threshold and scoring-cost measurements (Exp#2–5) and the conditional-information grids are archived there, and no version-2 claim rests on them.

Exp#7: Offline and in-loop targets differ. The archived offline two-view AUC is 0.948, while on the served-oracle calibration set the reconstructed scorer reaches 0.643 ($[0.632, 0.655]$, per-layer 0.54–0.76). We report the drop but

do not rely on it: the bridge of §IV-B reproduces neither its size nor its direction on version-2 traces, and target, trajectory and feature realization change together here, so its cause is unidentified. The served oracle of a decode step is, per layer, the $K = \min\{n, \max(32, \lfloor 0.1n \rfloor)\}$ blocks with the largest unmasked attention at that step; a policy’s miss is the share of them outside its kept set of $B = \max(1, \text{round}(0.2n))$ blocks, mandatory blocks included. On four LongBench tasks at 20% retention (48 requests each) the H2O-style accumulator has request-mean served-oracle miss 0.579 against 0.623 for the reconstructed scorer (paired difference $[0.033, 0.055]$) while the per-task F1 intervals all overlap; the confirmatory comparison is Exp#8. A budget below the oracle set forces a miss of at least $F = \max(0, 1 - B/K)$ at that step. From the logged layer-mean counts, averaged over steps and then requests as the miss is, the 4K-cap floor is 0.219 ($K = 32$ at every step, $B \leq 26$), leaving residuals 0.360 and 0.404. The floor is identical for the two policies on every request, so subtracting it leaves the paired contrast unchanged algebraically: it shows how much of the absolute miss no selector could avoid and is not an independent robustness check. A feedback-shift check and the other caps and budget fractions are in the artifact.

IX. LIMITATIONS AND ARTIFACT AUDIT

External validity. Historical traces cover four instruction-tuned 7–8B families; corrected v2 traces and the pinned rerun cover two, at up to about 4K context. The archived 16K Llama and short-context Qwen2.5-14B runs are separate checks, not v2 replications. Nothing is extrapolated to MoE, base checkpoints, 32K contexts or sampling; decoding is greedy.

Query-coordinate audit. The legacy extractor hooks `q_proj` before rotary transformation but reads post-rotation keys, so it never computed the attention-space query-key product; the old dot-max probe is no faithful Quest control and we claim no intrinsic query redundancy. Version 2 aligns queries and keys after RoPE (a one-head test reproduces the attention probabilities from its scaled dot products); even so a dot-max probe is not Quest’s page-bound algorithm. Adding it to the two magnitude views changes held-out *request-mean* AUC by -0.0070 on the 128 Llama prompts (32 test requests, all negative; $[-0.0082, -0.0058]$; two-view mean 0.899) and by $+0.0020$ on the matched Qwen prompts (31/32 positive; $[0.0016, 0.0024]$; mean 0.884): small, model-dependent, opposite in sign.

Generation, labels and the archived loop. The legacy collector refed the final prompt token and clamped unavailable horizons to the last recorded step; version 2 consumes the prefill-predicted token and generates genuine lookahead, so archived long-horizon results are not evidence for long-horizon generalization. The archived cells came from an unpinned float16 simulator whose Qwen2.5 path emits all-NaN logit rows (§VII), scored against one reference with articles removed before punctuation; the rerun pins the revision by hash, uses bfloat16 for both architectures and applies one memory patch (ingested attention weights replaced by a placeholder),

verified on a tiny model to change no statistic, mask or token. No rerun latency is reported; the GPUs were shared.

Tied scores and shared prompts. The archived AUPRC integrated one row at a time after a stable sort, crediting tied scores for row order; the two discrete probes cannot be recomputed, so Table II uses the threshold-based definition. The archived split held out model-requests, not source texts; version 2 adds a source-prompt split.

Inference, aggregation and targets. Information estimates depend on binning and conditional sparsity; overlapping marginal intervals are not an equivalence test; the primary TOST treats the seven dataset means as independent units, so it holds for this fixed collection of two architectures and seven tasks, exploratory samples do not replicate it, and non-rejection is not proof of no benefit. The collector averages attention with equal weight over all query heads and over a block’s actual members, whereas the loop averages head groups and pads ragged tail blocks (Table I), a difference we did not measure. The label is the attention four steps ahead while the loop re-decides every eight steps; a window target is untested. Both learned scorers minimize a pooled loss, so the reversal may belong to that objective and not to the cross-layer feature: a decision-balanced or within-decision ranking loss is untested. A budget shared across layers or requests makes cross-decision ordering the operating point, and offline candidates include the sink and recent blocks the loop reserves subject to its budget cap (Table I).

Parity levels of the reference backend. The committed gates (1024 tokens, four decode steps) pass their numeric tolerance. Extending them to four prompts from both datasets, 1024–4096 input tokens and 40 decode steps with repeated eviction separates three levels: the session reproduces native full-cache logits exactly (maximum error 0 in all 12 runs); but the compact and masked paths, fed the same teacher tokens and schedules, drift apart over successive evictions to a maximum logit difference of 0.66–1.44 in bfloat16 ($3.5\text{--}5.9\times$ the preset tolerance) and $3.6\times 10^{-5}\text{--}1.9\times 10^{-4}$ in float32 ($1.3\text{--}6.1\times$), so the extended test fails that tolerance. The four-order fall with precision fits floating-point ordering over 40 evictions but does not exclude a semantic difference; settling it needs a layer-by-layer comparison of retained indices, positions and K/V values against a higher-precision reference. Top-1 agreement is 972/984 gated steps in bfloat16 and 984/984 in float32. Tolerances were not raised.

Availability. The artifact contains LaTeX, result JSON, analysis code, predictor weights, tests, the version-2 collector, the reference validator and the rerun’s exact simulator source (MIT notice kept, verified against all 70 cells’ provenance hashes); model weights remain separately licensed. Code, records and figure sources are at <https://github.com/lzbaciz/KVSaliencyBench>; the default benchmark entry scores a prediction table without traces or weights. It always checks label counts, verifies candidate sets only against a supplied manifest, and cannot check how a scorer was trained. Raw traces are available on request.

X. RELATED WORK

Streaming learning studies evolving examples and distribution shift [12]; multi-view learning and relevance–redundancy analysis motivate comparing information sources [13], [14], which we apply to recorded attention dynamics without treating a small information estimate as a sufficiency proof. Our pooled-versus-within-decision split parallels user-weighted (grouped) AUC in click-through prediction [15], which averages per-user AUC by impressions because pooled AUC also rewards cross-user order; Eq. (1) weights by pairs instead. Calibration and conformal prediction [16]–[18] provide tools whose guarantees do not transfer to dependent runtime observations unchecked.

KV-management methods use heavy hitters, attention sinks with recent windows, recent observations, cross-layer signals, query bounds, adaptive allocation and learned retention [1]–[6], [19], [20]; PagedAttention [21] addresses physical serving. Our probes and allocation comparators are not interchangeable with those systems; this is an audit of the links between intermediate prediction and task quality, not a claim that these methods are unnecessary.

Closest to our question are prediction-first methods and evaluation suites for KV compression. AttentionPredictor [7] learns a lightweight model of next-token attention and uses it to identify critical tokens; we do not compare against it and ask instead which evaluation link a proxy’s ranking survives. Gao et al. [22] audit compression from the serving side, and Agrawal and Mayer [23] benchmark task quality, throughput, time-to-first-token and compression ratio jointly. SCBench [24] evaluates the whole KV-cache lifecycle, including shared-context reuse, and reports attention-distribution shift in long generation. Expected Attention [25] scores each KV pair by the attention expected from a distribution of future queries, itself a prediction of future attention, and releases KVPress, which standardizes more than twenty compression methods; KVzap [26] is a recent learned pruning method evaluated on such a suite, and Locret [6] trains retaining heads on a causal-importance target and validates them by end-task quality. Those suites and task-level benchmarks such as LongBench and RULER [11], [27] fix a common harness and ask whether a method is worth deploying; we fix the scorer and the label and vary the feature realization and the candidate set (§IV-B, Exp#7–8). The negative findings concern how proxies transfer to task quality in the tested implementations, not the value of learned or value-dependent selection [28].

XI. CONCLUSION

KV-saliency prediction and KV retention are related but distinct evaluation problems. On version-2 traces, the ranking advantage of learned scorers in pooled metrics reverses under per-decision top-decile budgets on request- and source-disjoint splits, less sharply at 20% retention. Pooled AUC is almost entirely cross-decision pairs; the logistic scorer’s binary cross-layer term raises it and lowers within-decision AUC and recall, promoting previous-layer-hot blocks that are positive less often than the blocks they displace. A separate pinned

bfloat16 masked-loop rerun finds the reconstructed scorer mean-F1 equivalent to the H2O-style accumulator within a practical ± 0.02 margin over seven datasets, as two complete runtime policies, both below full cache. The bridge puts reference-backend feature reconstruction within 0.003 AUC and leaves the archived served-oracle gap unexplained; the physical reference shows what storage a compact cache needs, not that it is equivalent to masking. These findings concern the tested data and implementations, not the impossibility of better query-aware selection. To evaluate a retention scorer, fix the candidate set and budget, measure inside each decision, then check answer quality on the policy’s own trajectory; the artifact’s default protocol measures the first two and verifies candidate sets when given a manifest.

ACKNOWLEDGMENT

Anthropic Claude (via Claude Code) assisted with manuscript drafting and the version-2 collector, physical-KV validator, audit scripts and decision-level benchmark protocol. OpenAI ChatGPT/Codex assisted with revisions to the abstract, introduction, evaluation diagram, statistical protocol, results text, limitations and conclusion, and with bootstrap, oracle-budget, reproducibility and PDF-check code. These tools generated or restructured text and code; the authors are responsible for verifying the retained content, experiments and conclusions.

REFERENCES

- [1] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen, “H2O: Heavy-hitter oracle for efficient generative inference of large language models,” in *Proc. Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 34 661–34 710.
- [2] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, “Efficient streaming language models with attention sinks,” in *Proc. International Conference on Learning Representations*, 2024.
- [3] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen, “SnapKV: LLM knows what you are looking for before generation,” in *Proc. Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 22 947–22 970.
- [4] J. Tang, Y. Zhao, K. Zhu, G. Xiao, B. Kasikci, and S. Han, “QUEST: Query-aware sparsity for efficient long-context LLM inference,” in *Proc. 41st International Conference on Machine Learning*, vol. 235. PMLR, 2024, pp. 47 901–47 911.
- [5] W. Lee, J. Lee, J. Seo, and J. Sim, “InfiniGen: Efficient generative inference of large language models with dynamic KV cache management,” in *Proc. 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, 2024, pp. 155–172.
- [6] Y. Huang, B. Yuan, X. Han, C. Xiao, and Z. Liu, “Locret: Enhancing eviction in long-context LLM inference with trained retaining heads on consumer-grade devices,” *Transactions on Machine Learning Research*, 2025.
- [7] Q. Yang, J. Wang, X. Li, Z. Wang, C. Chen, L. Chen, X. Yu, W. Liu, J. Hao, M. Yuan, and B. Li, “AttentionPredictor: Temporal patterns matter for KV cache compression,” in *Proc. Advances in Neural Information Processing Systems*, vol. 38, 2025, pp. 153 646–153 678.
- [8] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “LightGBM: A highly efficient gradient boosting decision tree,” in *Proc. Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [9] D. J. Schuurmann, “A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability,” *Journal of Pharmacokinetics and Biopharmaceutics*, vol. 15, no. 6, pp. 657–680, 1987.
- [10] D. Lakens, “Equivalence tests: A practical primer for t tests, correlations, and meta-analyses,” *Social Psychological and Personality Science*, vol. 8, no. 4, pp. 355–362, 2017.
- [11] Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, Z. Du, X. Liu, A. Zeng, L. Hou, Y. Dong, J. Tang, and J. Li, “LongBench: A bilingual, multitask benchmark for long context understanding,” in *Proc. 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 3119–3137.
- [12] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, “A survey on concept drift adaptation,” *ACM Computing Surveys*, vol. 46, no. 4, pp. 1–37, 2014, art. no. 44.
- [13] A. Blum and T. Mitchell, “Combining labeled and unlabeled data with co-training,” in *Proc. 11th Annual Conference on Computational Learning Theory*. ACM, 1998, pp. 92–100.
- [14] H. Peng, F. Long, and C. Ding, “Feature selection based on mutual information: Criteria of max-dependency, max-relevance, and min-redundancy,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 27, no. 8, pp. 1226–1238, 2005.
- [15] G. Zhou, X. Zhu, C. Song, Y. Fan, H. Zhu, X. Ma, Y. Yan, J. Jin, H. Li, and K. Gai, “Deep interest network for click-through rate prediction,” in *Proc. 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2018, pp. 1059–1068.
- [16] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. 34th International Conference on Machine Learning*, vol. 70. PMLR, 2017, pp. 1321–1330.
- [17] A. N. Angelopoulos and S. Bates, “Conformal prediction: A gentle introduction,” *Foundations and Trends in Machine Learning*, vol. 16, no. 4, pp. 494–591, 2023.
- [18] I. Gibbs and E. J. Candès, “Adaptive conformal inference under distribution shift,” in *Proc. Advances in Neural Information Processing Systems*, vol. 34, 2021, pp. 1660–1672.
- [19] Y. Feng, J. Lv, Y. Cao, X. Xie, and S. K. Zhou, “Ada-KV: Optimizing KV cache eviction by adaptive budget allocation for efficient LLM inference,” in *Proc. Advances in Neural Information Processing Systems*, vol. 38, 2025, pp. 125 462–125 498.
- [20] Z. Cai, Y. Zhang, B. Gao, Y. Liu, Y. Li, T. Liu, K. Lu, W. Xiong, Y. Dong, J. Hu, and W. Xiao, “PyramidKV: Dynamic KV cache compression based on pyramidal information funneling,” in *Proc. Conference on Language Modeling*, 2025.
- [21] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proc. 29th Symposium on Operating Systems Principles*. ACM, 2023, pp. 611–626.
- [22] W. Gao, X. Zhou, P. Sun, T. Zhang, and Y. Wen, “Rethinking key-value cache compression techniques for large language model serving,” in *Proc. Machine Learning and Systems*, vol. 7, 2025.
- [23] N. Agrawal and R. Mayer, “Benchmarking KV-cache optimizations across task quality and system performance for long-context serving [Experiment, Analysis & Benchmark],” Preprint, Univ. Bayreuth EPub, 2026. [Online]. Available: https://doi.org/10.15495/EPub_UBT_00009365
- [24] Y. Li, H. Jiang, Q. Wu, X. Luo, S. Ahn, C. Zhang, A. H. Abdi, D. Li, J. Gao, Y. Yang, and L. Qiu, “SCBench: A KV cache-centric analysis of long-context methods,” in *Proc. International Conference on Learning Representations*, 2025.
- [25] A. Devoto, M. Jeblick, and S. Jégou, “Expected attention: KV cache compression by estimating attention from future queries distribution,” *arXiv preprint arXiv:2510.00636*, 2025.
- [26] S. Jégou and M. Jeblick, “KVzap: Fast, adaptive, and faithful KV cache pruning,” *arXiv preprint arXiv:2601.07891*, 2026.
- [27] C.-P. Hsieh, S. Sun, S. Kriman, S. Acharya, D. Rekesh, F. Jia, Y. Zhang, and B. Ginsburg, “RULER: What’s the real context size of your long-context language models?” in *Proc. Conference on Language Modeling*, 2024.
- [28] Z. Guo, H. Kamigaito, and T. Watanabe, “Attention score is not all you need for token importance indicator in KV cache reduction: Value also matters,” in *Proc. 2024 Conference on Empirical Methods in Natural Language Processing*, 2024, pp. 21 158–21 166.